

MANUAL TÉCNICO

Conector de Base de Datos

Formulario Web con Node.js y SQLite

Materia: Sistemas de Información

Docente: Domingo Britos

Integrantes: Francisco Verdi, Joaquín Chapini, Valentín Agüero

Fecha de entrega: 1 de octubre de 2026

1. Introducción: ¿qué es un conector de base de datos?

Un conector de base de datos es la pieza de software que permite que una aplicación, en este caso, un formulario web, se comunique con una base de datos para guardar, leer, modificar o eliminar información.

Un formulario HTML, por sí solo, no puede conectarse directamente a una base de datos. Esto es una decisión de seguridad de los navegadores, ya que el código que corre dentro de una página web (JavaScript en el navegador) no tiene permiso para leer ni escribir archivos del sistema. Si lo tuviera, cualquier sitio web podría acceder o dañar archivos de la computadora del usuario sin su consentimiento.

Por eso, siempre que una página web necesita guardar información de forma permanente, existe un servidor en el medio: un programa aparte que sí tiene permiso para trabajar con archivos y bases de datos, y que recibe pedidos desde el navegador a través de internet.

En este proyecto, el conector está formado por dos piezas de software trabajando juntas:

- Node.js, que permite ejecutar JavaScript fuera del navegador, como un programa de servidor.
- better-sqlite3, una librería que le permite a Node.js leer y escribir en un archivo de base de datos SQLite.

2. ¿Por qué SQLite?

Por que la mayoría requiere instalar un programa aparte (MySQL, PostgreSQL, SQL Server, MongoDB, etc.), configurarlo como un servicio, crear usuarios y contraseñas, y dejarlo corriendo en segundo plano.

En cambio, SQLite es distinto ya que no es un programa que se instala y se prende, sino una librería (un pedacito de código) que lee y escribe un único archivo. Ese archivo (en este proyecto, database.db) contiene toda la base de datos: tablas, columnas y filas. Sin necesidad de tener un servidor de base de datos aparte (con usuarios, contraseñas, etc.)

3. Arquitectura de la solución

El flujo de datos, desde que el usuario escribe algo en el formulario hasta que queda guardado (o consultado), es el siguiente:

```
[Usuario completa el formulario en el navegador]
|
|
[JavaScript del navegador (js.js): recolecta los datos con "fetch"]
| (los envia por internet, en formato JSON)
|
[Servidor Node.js + Express (server.js): recibe el pedido]
|
|
[better-sqlite3: ejecuta la instrucción SQL de guardado o consulta]
|
|
[Archivo database.db: los datos quedan guardados de forma permanente]
```

4. Requisitos previos

Antes de empezar, es necesario tener instalado Node.js (versión 18 o superior), descargable desde nodejs.org, eligiendo la versión “LTS”.

Para comprobar que está instalado, se abre una terminal (CMD o PowerShell en Windows, Terminal en Mac) y se escribe:

```
node -v
```

Qué hace: Si devuelve un número de versión (por ejemplo v20.11.0), Node ya está listo para usar.

5. Estructura del proyecto

El proyecto se organiza en carpetas para separar lo que ve el usuario de lo que corre en el servidor:

```
Conector-de-Base-de-Datos/      (la carpeta del proyecto)
├── node_modules/                (se genera automáticamente)
├── package.json                 (se genera automáticamente)
├── package-lock.json            (se genera automáticamente)
├── server.js                    <- código del servidor
├── database.db                  <- se crea sola al ejecutar el servidor (en donde están los datos)
└── public/
    ├── index.html               <- el formulario
    ├── js.js                    <- lógica del formulario
    └── ver-datos.html            <- página para consultar los datos guardados
```

¿Por qué separamos public (en donde va ubicado el HTML y el Java) del resto? Porque el servidor está configurado para mostrar únicamente lo que hay dentro de public al usuario que visita la página. El código del servidor (server.js) y la base de datos (database.db) quedan fuera del alcance del navegador: para que nadie pueda acceder a ellos escribiendo una URL, solo el propio servidor puede leerlos y escribirlos.

6. Instalación de dependencias

Parados con la terminal dentro de la carpeta Conector-de-Base-de-Datos, se ejecutan dos comandos.

6.1 Inicializar el proyecto

```
npm init -y
```

Qué hace: Crea el archivo “package.json”: registra su nombre y qué herramientas externas utiliza.

6.2 Instalar las herramientas necesarias

```
npm install express better-sqlite3
```

Qué hace: Descarga dos librerías: express, que simplifica la creación de un servidor web, y better-sqlite3, que permite que Node.js lea y escriba archivos de base de datos SQLite. Se guardan en una carpeta node_modules que se genera sola.

7. El código del servidor: server.js

Este archivo va ubicado fuera de “public”. A continuación, la explicación del código del server:

Conexión con la base de datos:

```
const express = require('express');
const Database = require('better-sqlite3');

const app = express();
const db = new Database('database.db');
```

Qué hace: Se importan las dos librerías instaladas. “app” representa al servidor que se va a construir. “db” abre (o crea, si no existe todavía) el archivo database.db. Esta línea es todo lo que hace falta para conectar con la base de datos.

Configuración básica del servidor:

```
app.use(express.json());
app.use(express.static('public'));
```

Qué hace: La primera línea indica que el servidor sepa interpretar los datos que le llegan en formato JSON. La segunda indica que muestre automáticamente cualquier archivo ubicado en la carpeta public (como index.html o ver-datos.html) a quien visite la página.

Creación de la tabla:

```
db.exec(`
  CREATE TABLE IF NOT EXISTS respuestas (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT,
    apellido TEXT,
    curso TEXT,
    division TEXT,
    dni TEXT,
    telefono TEXT,
    modalidad TEXT,
    deporte TEXT,
    buffet TEXT
  )
`);
```

Qué hace: Crea la tabla respuestas dentro de la base de datos, con una columna por cada dato que pide el formulario. IF NOT EXISTS evita un error si la tabla ya fue creada en una ejecución anterior. Se ejecuta cada vez que arranca el servidor.

Ruta para GUARDAR datos (INSERT):

```
app.post('/api/guardar', (req, res) => {
  const { nombre, apellido, curso, division, dni, telefono, modalidad, deporte, buffet } =
    req.body;

  const stmt = db.prepare(`
    INSERT INTO respuestas (nombre, apellido, curso, division, dni, telefono, modalidad,
    deporte, buffet)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
  `);
  stmt.run(nombre, apellido, curso, division, dni, telefono, modalidad, deporte, buffet);

  res.json({ ok: true });
});
```

Qué hace: Define una “puerta de entrada” (ruta) en /api/guardar. Cuando el formulario envía datos hacia ahí, el servidor los recibe en req.body, prepara una instrucción SQL de tipo INSERT y la ejecuta con los valores recibidos.

Los signos de interrogación (?) son reemplazados en orden por cada dato: esta forma de escribirlo evita que alguien inyecte código malicioso a través del formulario. El servidor responde { ok: true } para avisar que todo salió bien.

Ruta para EXTRAER / CONSULTAR datos (SELECT):

```
app.get('/api/respuestas', (req, res) => {
  const filas = db.prepare('SELECT * FROM respuestas').all();
  res.json(filas);
});
```

Qué hace: Define otra ruta, esta vez para consultar los datos guardados en lugar de insertarlos. “SELECT * FROM respuestas” significa “traeme todas las columnas de todas las filas de la tabla respuestas”. Esta ruta es la que usa la página ver-datos.html para mostrar la información ya cargada, y también puede visitarse directamente en “http://localhost:3000/api/respuestas” para ver el resultado en formato JSON.

Encender el servidor:

```
app.listen(3000, () => {
  console.log('Servidor corriendo en http://localhost:3000');
});
```

Qué hace: Prende el servidor y lo deja escuchando pedidos en el puerto 3000 de la computadora. Mientras esta instrucción esté corriendo, el proyecto está disponible en “http://localhost:3000”.

8. El código del formulario: js.js

Este archivo va ubicado dentro de public, junto al index.html.

La “mochila” de datos y las referencias al HTML:

```
let datosUsuario = {};

const formulario = document.getElementById('formulario');
const preguntas = document.getElementById('Preguntas');
const final = document.getElementById('final');
const resultado = document.getElementById('resultado');
```

Qué hace: Se crea un objeto vacío (datosUsuario) que funciona como un “baúl” o una “mochila”; a medida que el usuario avanza por los formularios de la página, se van guardando ahí los datos que completa. Luego se guardan referencias a cada formulario, usando sus id del HTML, para poder manipularlos desde el código.

Envío del primer formulario:

```
formulario.addEventListener('submit', (e) => {
  e.preventDefault();

  datosUsuario.nombre = document.getElementById('nombre').value;
  datosUsuario.apellido = document.getElementById('Apellido').value;
  datosUsuario.curso = document.getElementById('curso').value;
  datosUsuario.division = document.getElementById('Division').value;

  formulario.hidden = true;
  preguntas.hidden = false;
});
```

Qué hace: Se “escucha” el envío (submit) del primer formulario. “e.preventDefault()” evita que la página se recargue, sino, el navegador continua con su comportamiento por defecto de un formulario HTML que haría perder los datos. Se leen los valores escritos por el usuario con “.value” y se guardan en la mochila. Por último, se oculta el primer formulario y se muestra el segundo.

Envío del segundo formulario y guardado en el servidor:

```
preguntas.addEventListener('submit', async (e) => {
  e.preventDefault();

  datosUsuario.dni = document.getElementById('dni').value;
  datosUsuario.telefono = document.getElementById('telefono').value;
  datosUsuario.modalidad = document.querySelector('input[name="Modalidad"]:checked')?.value ||
  '';
  datosUsuario.deporte = document.querySelector('input[name="deporte"]:checked')?.value || '';
  datosUsuario.buffet = document.querySelector('input[name="Buffet"]:checked')?.value || '';

  try {
    const respuesta = await fetch('/api/guardar', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(datosUsuario)
    });

    const data = await respuesta.json();

    preguntas.hidden = true;
    final.hidden = false;

    resultado.textContent = data.ok
      ? 'Gracias! Tus respuestas fueron guardadas correctamente.'
      : 'Hubo un problema al guardar los datos.';
  } catch (error) {
    preguntas.hidden = true;
    final.hidden = false;
    resultado.textContent = 'Error de conexion con el servidor.';
  }
});
```

Qué hace: Se completa la mochila con los datos del segundo formulario. Para los radio buttons se usa “document.querySelector(“input[name=“...“]:checked”)”, que busca cuál opción está tildada; el “?.value”, evita un error si no se tildó ninguna. Luego, “fetch” envía la mochila completa al servidor mediante “POST”, en formato JSON, a la ruta “/api/guardar”. El bloque “try/catch” protege el código: si el servidor no responde, se le avisa al usuario en vez de dejar la página colgada.

9. Consultar los datos guardados (ver-datos.html)

Además de guardar información, debemos de saber como extraer los datos ya cargados en la base. Para eso se agrega una página nueva, ver-datos.html, ubicada también dentro de public, que muestra en una tabla todo lo que hay guardado hasta el momento.

Función que trae y muestra los datos:

```
async function cargarDatos() {
  const respuesta = await fetch('/api/respuestas');
  const datos = await respuesta.json();

  document.getElementById('contador').textContent =
    `Total de registros: ${datos.length}`;

  const cuerpoTabla = document.getElementById('cuerpoTabla');
```

```

cuerpoTabla.innerHTML = '';

datos.forEach(fila => {
  const tr = document.createElement('tr');
  tr.innerHTML = `
    <td>${fila.id}</td>
    <td>${fila.nombre}</td>
    <td>${fila.apellido}</td>
    <td>${fila.curso}</td>
    <td>${fila.division}</td>
    <td>${fila.dni}</td>
    <td>${fila.telefono}</td>
    <td>${fila.modalidad}</td>
    <td>${fila.deporte}</td>
    <td>${fila.buffet}</td>
  `;
  cuerpoTabla.appendChild(tr);
});
}

cargarDatos();

```

Qué hace: Apenas se carga la página, “cargarDatos()” le pide al servidor (con fetch) la lista completa de respuestas usando la ruta “/api/respuestas” que ya existía en server.js. Por cada registro recibido, arma una fila de tabla (<tr>) con sus datos y la agrega al “<tbody>” de la página. Así, en lugar de un JSON crudo, se ve una tabla ordenada con toda la información cargada hasta el momento, incluyendo la que se va agregando por cada registro. ¿Porqué “tbody” en vez de “body”? Por que “tbody”, agrupa las filas de datos de una tabla.

Para acceder a esta página, no hace falta modificar server.js, podemos visitar:

<http://localhost:3000/ver-datos.html>

10. Cómo ejecutar el proyecto

1. Abrir una terminal ubicada en la carpeta convector-basedatos (dentro de la carpeta del proyecto).
2. Ejecutar: node server.js
3. Debería aparecer el mensaje “Servidor corriendo en http://localhost:3000”.
4. Abrir un navegador y visitar “http://localhost:3000”.
5. Completar el formulario y enviarlo.
6. Para ver todo lo guardado, visitar <http://localhost:3000/ver-datos.html>.

11. Video Tutorial

<https://youtu.be/jD2fspTEjII>